PROGRAMACIÓN ORIENTADA A OBJETOS · TRABAJO PRÁCTICO OBLIGATORIO

Sistema de Gestión para Guarderías de Mascotas

Entrega Parcial 1: propuesta de negocio y diagrama de clases preliminar

CONTENIDO

1. Propuesta de negocio
2. Diagrama de clases preliminar
3. Relaciones entre clases
4. Patrones y buenas prácticas (GRASP y SOLID)

Lenguaje	Java
Fecha	Octubre de 2026

1. Propuesta de negocio

Problema

Las guarderías y hoteles de mascotas suelen gestionar sus reservas por mensajería y registros en papel. Esto genera sobreventa de lugares en temporada alta, falta de control sanitario al ingreso (no se verifica de forma sistemática que cada animal tenga sus vacunas o controles al día) y poca información para los dueños sobre lo que ocurre durante la estadía. Cuando el negocio tiene más de una sede, nadie sabe con certeza cuántos lugares libres hay en cada sucursal.

Para quién

Para guarderías caninas, hoteles de mascotas y veterinarias con servicio de hospedaje, con una o varias sucursales. Los usuarios del sistema son los empleados del establecimiento.

Solución propuesta

Un sistema de escritorio que centraliza la operación del negocio. Cada sucursal tiene sus alojamientos, preparados para una especie (perros, gatos, conejos o aves) y de categoría estándar o premium, con un recargo en el precio. Cada especie tiene sus propios requisitos de ingreso: perros y gatos deben tener sus vacunas obligatorias vigentes, mientras que conejos y aves necesitan un control veterinario reciente. El sistema los valida automáticamente en el check-in. Durante la estadía se lleva una bitácora diaria de comidas, medicación e incidentes, y ante un incidente se avisa al dueño y al encargado de la sucursal.

Funcionalidades principales

- Búsqueda de alojamientos disponibles por sucursal
- Gestión de reservas de hospedaje
- ABM de clientes
- ABM de mascotas
- ABM de empleados
- ABM de alojamientos
- Check-in con control sanitario
- Check-out y cobro de la estadía
- Bitácora diaria con avisos al dueño
- Control de ocupación por sucursal

2. Diagrama de clases preliminar

El modelo tiene **21 elementos**: **14 clases** (2 abstractas, 1 clase interna y 2 clases anidadas estáticas), **3 interfaces** y **4 enumeraciones**, organizados en 6 paquetes.

Paquete	Contenido
`modelo.persona`	Persona *(abstracta)*, Cliente, Empleado
`modelo.mascota`	Mascota *(abstracta)*, Perro, Gato, Conejo, Ave, Mascota.Vacuna *(clase anidada estática)*
`modelo.sucursal`	Sucursal, Alojamiento
`modelo.reserva`	Reserva, Reserva.Pago *(clase anidada estática)*, Reserva.RegistroDiario *(clase interna)*
`modelo.interfaces`	Facturable, Vacunable, Notificable
`modelo.enums`	Especie, Categoria, EstadoReserva, TipoRegistro

Cómo leer los diagramas

Primero se muestra una vista general con todas las clases y sus relaciones; después, una vista detallada por paquete con atributos y métodos. El diagrama completo con todo el detalle se entrega además como archivo de imagen aparte.

Notación	Significado
Flecha con triángulo vacío	Herencia: la flecha apunta a la clase padre
Línea punteada con triángulo	Realización: la clase implementa la interfaz
Rombo relleno	Composición: la parte no existe sin el todo (el rombo va del lado del todo)
Rombo vacío	Agregación: la parte existe por su cuenta
Flecha simple	Asociación navegable
Flecha punteada	Dependencia: la clase usa a la otra dentro de un método
Nombre en cursiva	Clase o método abstracto
`-` `#` `+`	Visibilidad privada, protegida y pública
Recuadro gris	En las vistas por paquete: clase de otro paquete, mostrada solo como referencia

Figura 1. Vista general del modelo

Las 21 clases, interfaces y enumeraciones con sus relaciones y cardinalidades. Los atributos y métodos se detallan en las figuras 2 a 5.

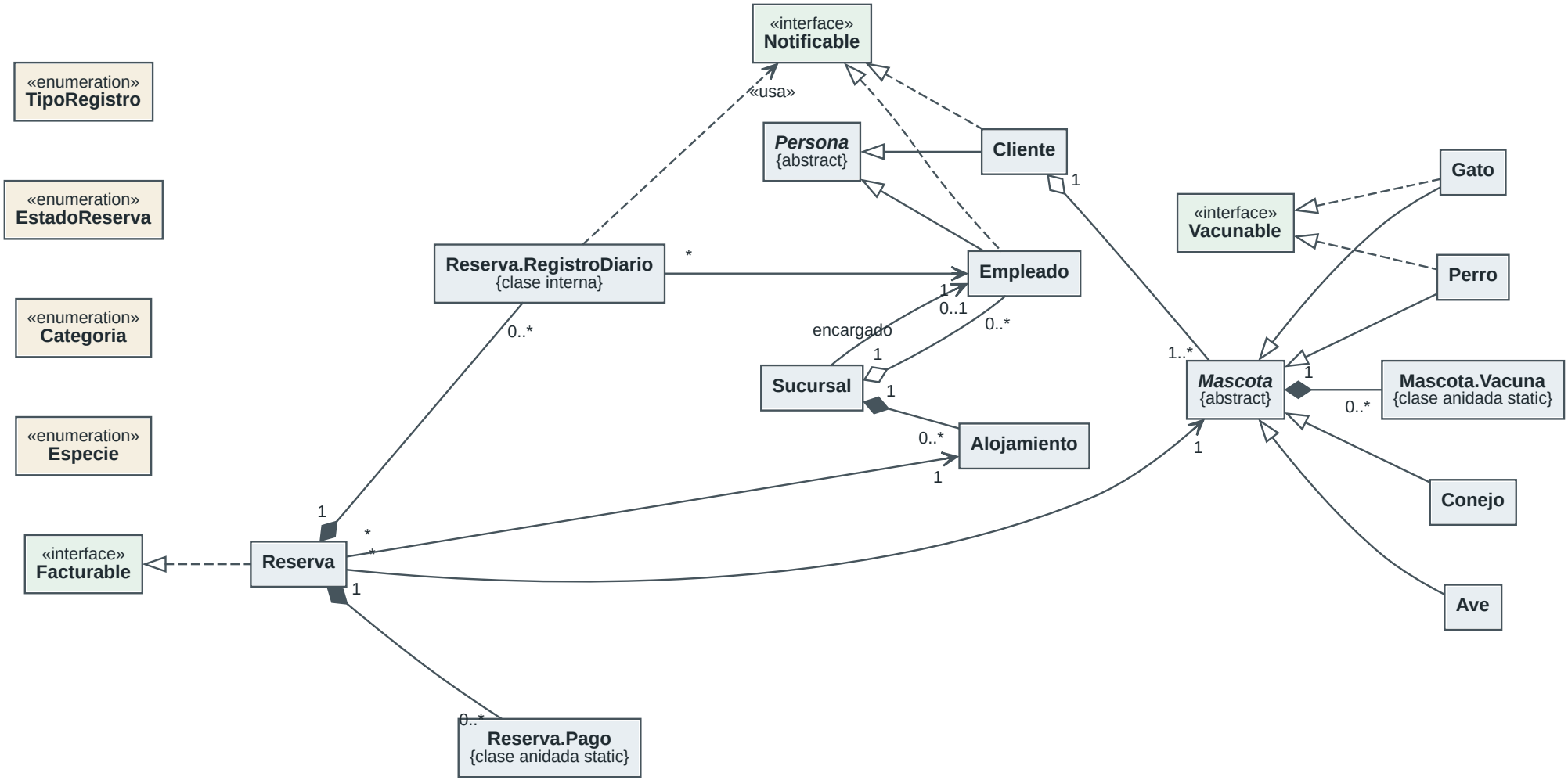


Figura 2. Personas

Persona es abstracta y se especializa en Cliente y Empleado. Ambas implementan Notificable, cada una a su manera: el cliente recibe avisos sobre su mascota y el empleado, avisos internos.

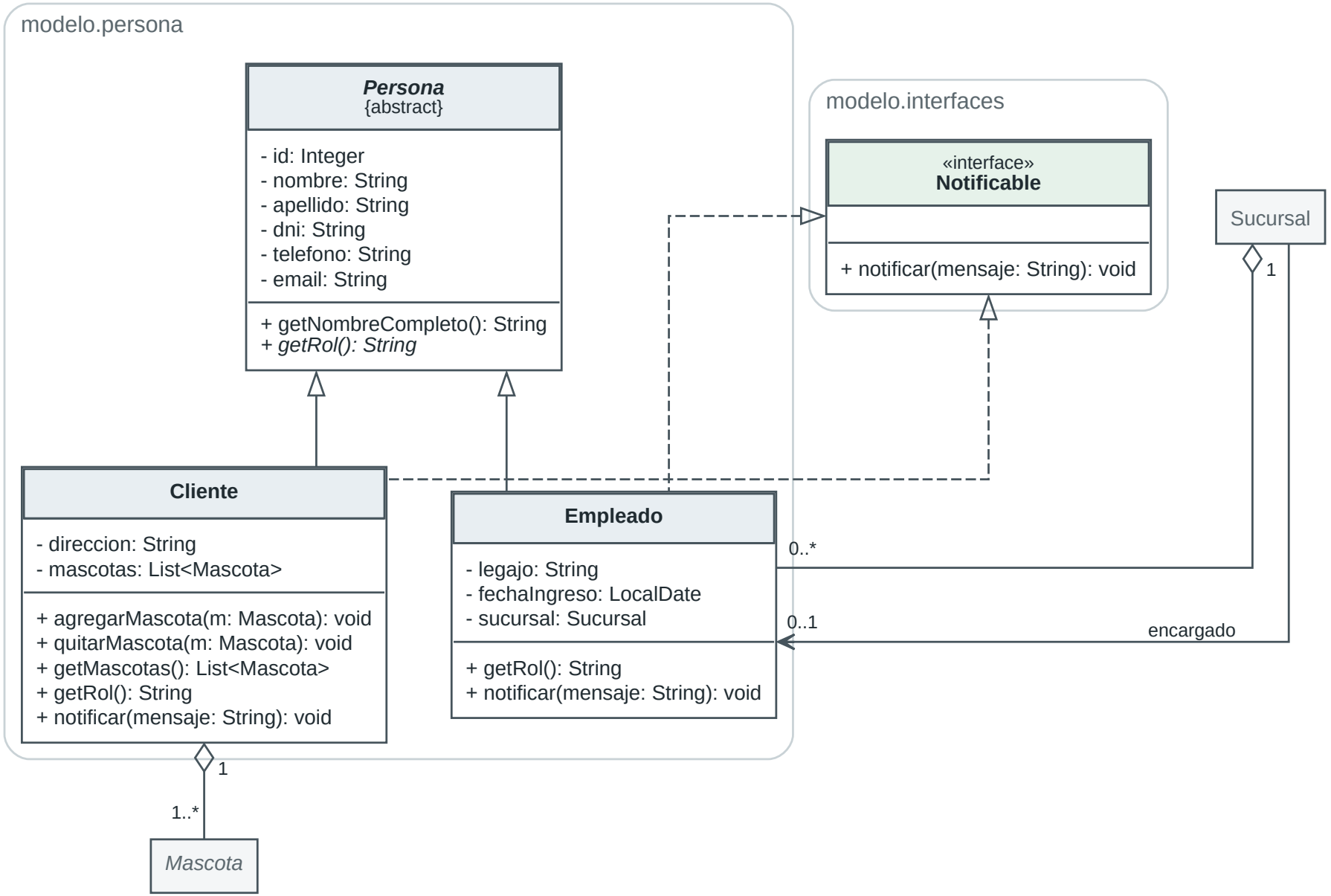


Figura 3. Mascotas

Mascota es abstracta: cada especie implementa validarIngreso() a su manera. Solo Perro y Gato implementan Vacunable. Vacuna es una clase anidada estática de Mascota.

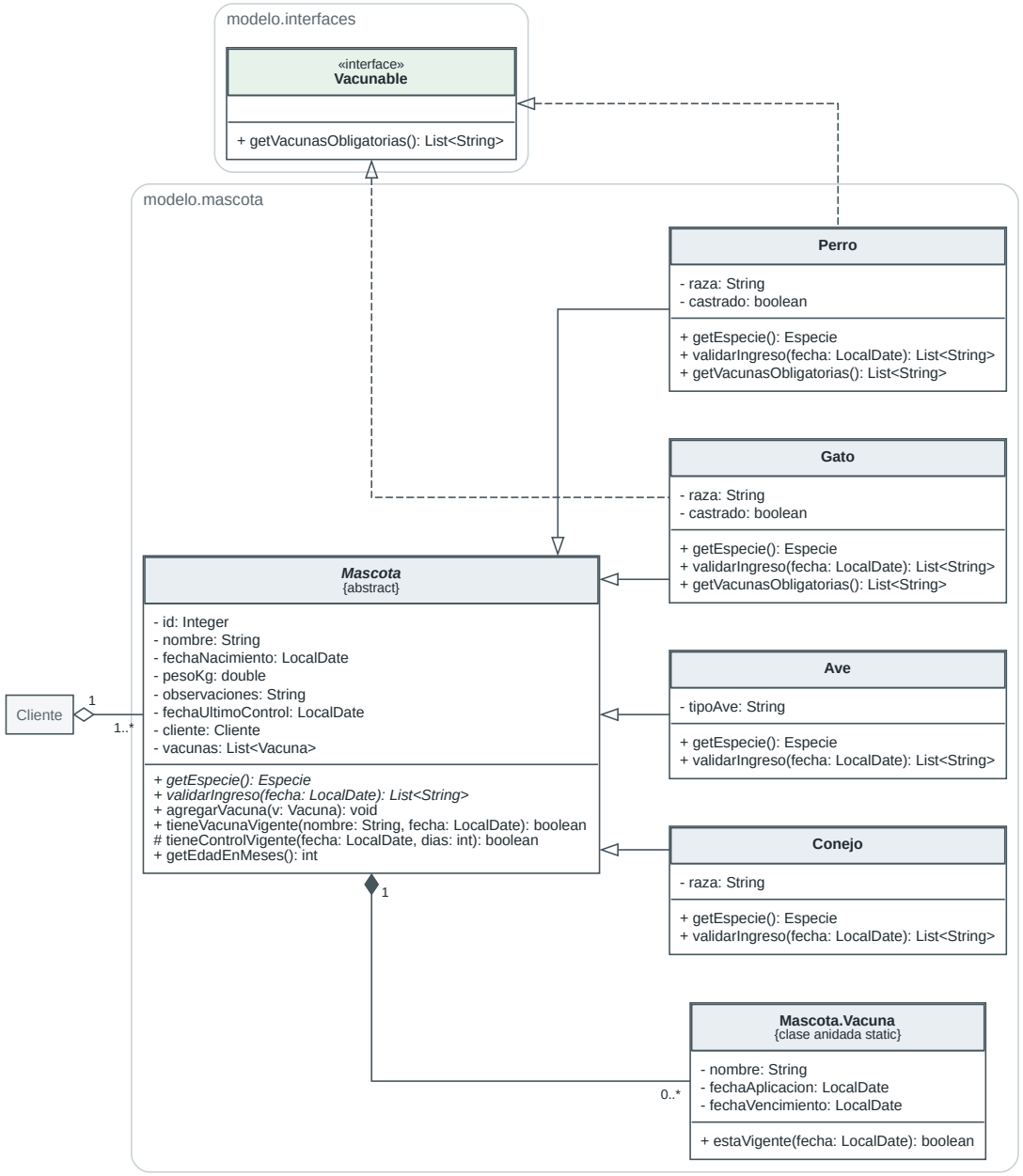


Figura 4. Sucursales y alojamientos

Cada sucursal compone sus alojamientos y agrupa a sus empleados; uno de ellos es el encargado. Cada alojamiento admite una especie y tiene una categoría que aplica un recargo al precio.

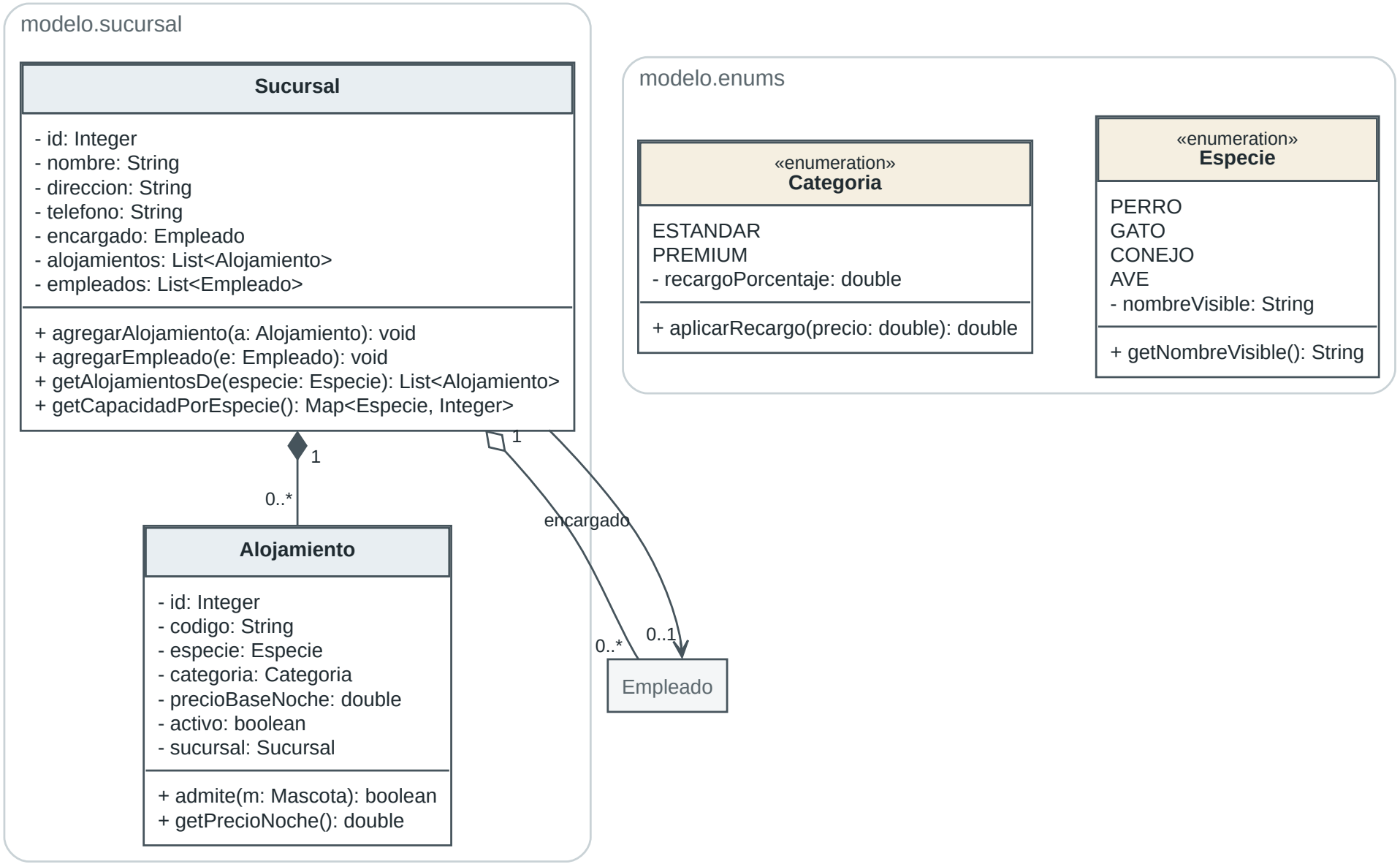
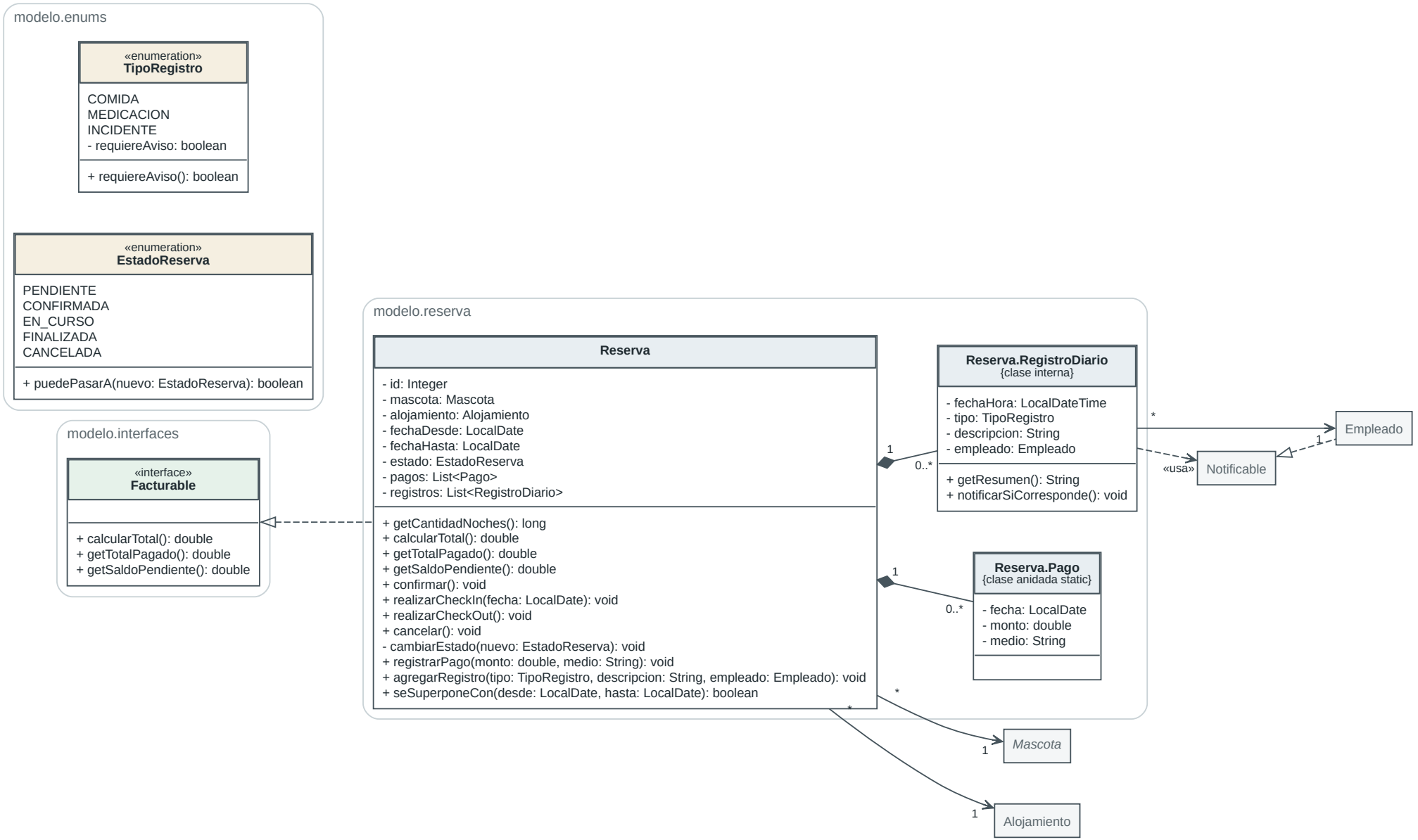


Figura 5. Reservas

Reserva implementa Facturable. Pago es una clase anidada estática y RegistroDiario una clase interna, porque necesita los datos de su reserva para avisar al cliente y al encargado.



3. Relaciones entre clases

Criterio utilizado: **composición** cuando la parte no tiene sentido sin el todo y se elimina con él; **agregación** cuando la parte existe por su cuenta; **asociación** cuando un objeto conoce a otro y lo guarda como atributo; **dependencia** cuando solo lo usa dentro de un método.

Herencia (generalización y especialización)

Clase padre (abstracta)	Clases hijas	Qué cambia en cada hija
Persona	Cliente, Empleado	El cliente tiene mascotas y dirección; el empleado tiene legajo y sucursal. Cada uno define su rol y su forma de recibir avisos
Mascota	Perro, Gato, Conejo, Ave	Los requisitos de ingreso: vacunas obligatorias para perros y gatos, control veterinario reciente para conejos y aves

Realización de interfaces

Interfaz	La implementan	Para qué
Facturable	Reserva	Calcular el total, lo pagado y el saldo
Vacunable	Perro, Gato	Solo las especies que requieren vacunas
Notificable	Cliente, Empleado	Recibir avisos, cada uno a su manera

Asociación, agregación, composición y dependencia

Desde	Hacia	Tipo	Cardinalidad	Justificación
Cliente	Mascota	Agregación	1 a 1..*	La mascota tiene sus propios datos e historial
Mascota	Mascota.Vacuna	Composición	1 a 0..*	Una vacuna registrada no existe sin su mascota
Sucursal	Alojamiento	Composición	1 a 0..*	Un lugar no existe fuera de su sucursal
Sucursal	Empleado	Agregación	1 a 0..*	El empleado existe aunque cambie de sucursal
Sucursal	Empleado (encargado)	Asociación	* a 0..1	Cada sucursal tiene un encargado que recibe los avisos
Reserva	Mascota	Asociación	* a 1	Una mascota tiene muchas reservas a lo largo del tiempo
Reserva	Alojamiento	Asociación	* a 1	Ídem para el alojamiento
Reserva	Reserva.Pago	Composición	1 a 0..*	Permite pagar en partes (seña y saldo)
Reserva	Reserva.RegistroDiario	Composición	1 a 0..*	La bitácora pertenece a esa estadía
RegistroDiario	Empleado	Asociación	* a 1	Quién cargó el registro
RegistroDiario	Notificable	Dependencia	—	Avisa a quien corresponda sin saber si es cliente o empleado

4. Patrones y buenas prácticas

El diseño aplica los patrones GRASP y los principios SOLID vistos en clase. Los tres principales son:

1. **Polimorfismo (GRASP):** cada especie implementa `validarIngreso()` a su manera, y `Cliente` y `Empleado` implementan `notificar()` cada uno a la suya. Ningún `if` pregunta por el tipo.
2. **Experto en información (GRASP):** cada responsabilidad la tiene la clase que posee los datos. `Reserva` calcula su total, `Alojamiento` decide si admite a una mascota y `Vacuna` sabe si está vigente.
3. **Segregación de interfaces (SOLID):** `Vacunable` la implementan solo `Perro` y `Gato`, de modo que conejos y aves no están obligados a tener métodos de vacunas que no usan.

GRASP

Patrón	Dónde se aplica
Experto en información	`Reserva.calcularTotal()`, `Alojamiento.admite()`, `Mascota.validarIngreso()`, `Vacuna.estaVigente()`
Creador	`Reserva` crea sus pagos y sus registros diarios; `Mascota` crea sus vacunas
Bajo acoplamiento	`RegistroDiario` solo conoce la interfaz `Notificable`, no a `Cliente` ni a `Empleado`
Alta cohesión	Cada clase tiene un solo foco (por ejemplo, `Vacuna` solo sabe de su vigencia y `Categoria` solo del recargo)
Polimorfismo	`validarIngreso()`, `notificar()`, `getRol()`
Variaciones protegidas	La forma de avisar queda detrás de `Notificable`; las reglas sanitarias de cada especie quedan encapsuladas en su propia clase

SOLID

Principio	Dónde se aplica
S · Responsabilidad única	`Reserva` gestiona la estadía, `Vacuna` su vigencia y `Categoria` el recargo: cada clase tiene una sola razón para cambiar
O · Abierto/cerrado	Sumar una especie (por ejemplo, hurones) es crear una clase hija sin modificar las demás; una categoría nueva es un valor más del enum
L · Sustitución de Liskov	Cualquier `Mascota` o `Persona` puede usarse donde se espera la clase padre
I · Segregación de interfaces	`Vacunable` la implementan solo perros y gatos: conejos y aves no están obligados a tener métodos de vacunas
D · Inversión de dependencias	`RegistroDiario` avisa a una lista de `Notificable` sin depender de las clases concretas `Cliente` y `Empleado`

Otros conceptos aplicados en el modelo

- **Encapsulamiento:** atributos privados; el estado de una reserva solo cambia por `cambiarEstado()`, que valida la transición con `EstadoReserva.puedePasarA()`.
- **Clases internas y anidadas:** `Reserva.RegistroDiario` (interna), `Mascota.Vacuna` y `Reserva.Pago` (anidadas estáticas).
- **Enumeraciones con atributos y métodos:** `Categoria.aplicarRecargo()`, `EstadoReserva.puedePasarA()`, `TipoRegistro.requiereAviso()`.
- **Colecciones:** `List` para mascotas, vacunas, alojamientos, empleados, pagos y registros; `Map<Especie, Integer>` para la capacidad de cada sucursal.
- **Datos wrapper:** los identificadores son `Integer`, que vale `null` mientras el objeto todavía no tiene un identificador asignado.